

Algorytmy współpracy i rywalizacji w systemach wieloagentowych

Jakub Jaśków, Justyna Ziemichód

1 sierpnia 2026

Plan prezentacji

- 1 Wprowadzenie i architektury MAS
- 2 Od klasycznego AI do MARL
- 3 Aparat matematyczny
- 4 Algorytmy współpracy
- 5 Algorytmy rywalizacji
- 6 Metaheurystyki i algorytmy ewolucyjne
- 7 Ewaluacja i podsumowanie

Definicja agenta i systemu MAS

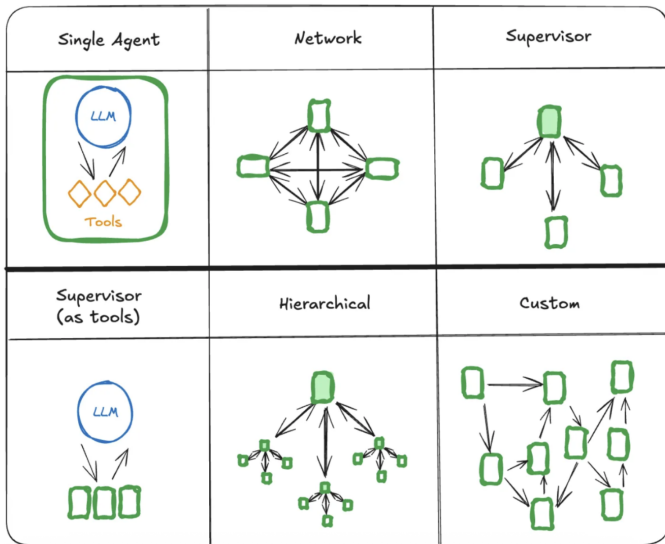
Definicja agenta: Autonomiczna jednostka obliczeniowa, która percepuje środowisko i wykonuje określone czynności w celu maksymalizacji własnej funkcji użyteczności.

System wieloagentowy (MAS – *Multi-Agent Systems*): Rozproszony układ autonomicznych jednostek. Jest to fundamentalna gałąź rozproszonej sztucznej inteligencji.

Główne zalety paradygmatu wieloagentowego:

- **Skalowalność:** Elastyczne dołączanie nowych węzłów do środowiska bez modyfikacji globalnego algorytmu.
- **Odporność na awarie:** Eliminacja pojedynczego punktu awarii.
- **Równoległość:** Możliwość jednoczesnego przetwarzania danych przez wiele niezależnych procesów.

Architektury wieloagentowe



Rysunek: Architektury wieloagentowe w bibliotece LangGraph.

Historycznie, systemy wieloagentowe wywodzą się z analitycznej teorii gier oraz klasycznej sztucznej inteligencji (AI – *Artificial Intelligence*).

Cechy klasycznego podejścia w MAS:

- Algorytmy optymalizują strategie wprost z równań matematycznych (np. algebraiczne poszukiwanie równowagi Nasha).
- Wymagają kompletnej wiedzy o modelu przejść środowiska.
- Sprawdzają się wyłącznie w zamkniętych środowiskach z niewielką i dyskretną przestrzenią stanów.

Ograniczenie: Wraz ze wzrostem liczby agentów występuje problem **kłątwy wymiarowości**. Równania stają się niemożliwe do analitycznego rozwiązania w czasie wielomianowym.

W celu rozwiązania problemu złożoności obliczeniowej, współczesne systemy wdrażają techniki uczenia maszynowego (ML – *Machine Learning*).

Multi-Agent Reinforcement Learning (MARL):

- Agenci nie dysponują modelem analitycznym a priori.
- Optymalna polityka decyzyjna jest aproksymowana (przybliżana) poprzez interakcję ze środowiskiem za pomocą algorytmów uczenia ze wzmocnieniem (RL – *Reinforcement Learning*).
- **Konkluzja:** Agenci MARL nadal dążą do osiągnięcia równowag znanych z teorii gier, jednak optymalizują je heurystycznie, wykorzystując sieci neuronowe do mapowania stanów na akcje na podstawie sygnałów nagrody.

Klasyfikacja algorytmów wieloagentowych

Podejście algorytmiczne zależy od pożądanego zachowania agentów. Główne rodziny algorytmów w MAS to:

- 1 **Algorytmy oparte na teorii gier (Rywalizacja):** Stosowane w środowiskach konfliktowych (np. zautomatyzowany handel). Przykłady: *Równowaga Nasha*, protokoły aukcyjne (Contract Net Protocol).
- 2 **Algorytmy koordynacji i konsensusu (Współpraca):** Zapewniają synchronizację niezależnych węzłów bez centralnego lidera. Przykłady: *protokoły konsensusu*, *rozproszona optymalizacja (DCOP)*, algorytmy roju (Flocking/Boids).
- 3 **Wieloagentowe uczenie ze wzmocnieniem (MARL):** Sieci neuronowe adaptujące się do otoczenia (np. Independent Q-Learning, czy algorytmy CTDE), stosowane, gdy wyliczenie analityczne jest niemożliwe.
- 4 **Algorytmy ewolucyjne i genetyczne (MAGA):** Populacje strategii agentów podlegają "mutacjom", a do rozwiązania złożonych problemów optymalizacyjnych przetrwają tylko te najlepiej dopasowane.

Model środowiska: wieloagentowa gra Markowa

Środowisko, w którym operują i wchodzi w interakcje agenci, definiuje się formalnie jako stochastyczną grę Markowa:

$$\mathcal{M} = \langle n, S, \mathcal{A}, T, R, \gamma \rangle$$

Formalna definicja elementów środowiska:

- n : Całkowita liczba agentów w systemie.
- S : Przestrzeń stanów środowiska (zbiór wszystkich możliwych sytuacji).
- $\mathcal{A}$: Połączona przestrzeń dostępnych akcji dla wszystkich agentów.
- T : Stochastyczna funkcja przejścia. Definiuje prawdopodobieństwo zmiany stanu środowiska na podstawie połączonego wektora akcji.
- R : Funkcja nagrody. Definiuje wartość liczbową przypisaną do wykonania danego przejścia między stanami.
- γ : Współczynnik dyskonta ($0 \leq \gamma < 1$). Parametr regulujący wagę nagród długoterminowych względem natychmiastowych.

Ewaluacja efektywności strategii agenta opiera się na równaniu Bellmana:

$$V_i^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_i(S_t, A_t, S_{t+1}) \mid S_0 = s \right]$$

Analiza czynników decyzyjnych:

- $V_i^\pi(s)$: Całkowita oczekiwana użyteczność stanu s przy implementacji przez agenta i strategii (polityki) π .
- $\mathbb{E}_\pi$: Wartość oczekiwana. Stanowi matematyczne uśrednienie wahań wyników wynikających ze stochastycznego charakteru środowiska.
- $\sum_{t=0}^{\infty} \gamma^t$: Zsumowanie nieskończonego szeregu kroków czasowych (t).
- R_i : Wartość natychmiastowej nagrody wygenerowanej dla agenta i .

Współpraca: algorytm konsensusu

Cel: Rozproszona synchronizacja stanu wszystkich węzłów bez udziału centralnego koordynatora.

$$x_i(k+1) = x_i(k) + \sum_{j \in N_i} a_{ij}(x_j(k) - x_i(k))$$

Analiza zmiennych równania:

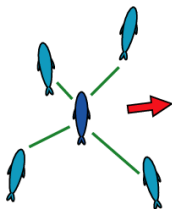
- $x_i(k+1), x_i(k)$: Wektor stanu agenta odpowiednio w kroku przyszłym oraz bieżącym.
- a_{ij} : Waga połączenia komunikacyjnego.
- $x_j(k) - x_i(k)$: Wektor błędu lokalnego. Agent iteracyjnie minimalizuje różnicę między stanem własnym a stanami sąsiadów, co prowadzi do globalnej zbieżności całego układu.

Przykład implementacji

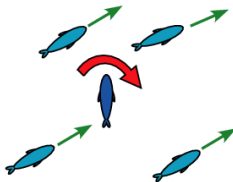
Robotyka rojowa: Algorytmy uśredniania prędkości w formacjach autonomicznych statków powietrznych.

Współpraca: algorytmy roju (Boids algorithms)

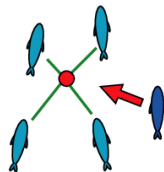
Cel: Koordynacja ruchu wielu agentów na podstawie lokalnych i biologicznie inspirowanych heurystyk, bez centralnego lidera.



(a)



(b)



(c)

Podstawowe zasady modelu Boids:

- a) Separacja (Separation)
- b) Wyrównanie (Alignment)
- c) Spójność (Cohesion)

Współpraca: rozproszona optymalizacja (DCOP)

Cel: Globalna optymalizacja decyzji w sieci poprzez minimalizację lokalnych konfliktów o zasoby (DCOP – *Distributed Constraint Optimization Problem*).

$$x^* = \arg \min_x \sum_{i,j \in E} f_{ij}(x_i, x_j)$$

Analiza zmiennych równania:

- x^* : Poszukiwany, ostateczny i optymalny plan działania dla wszystkich węzłów w sieci.
- E : Zbiór krawędzi grafu.
- $f_{ij}(x_i, x_j)$: Funkcja kary. Nakłada wysoki koszt, jeżeli decyzje (x_i, x_j) dwóch sąsiadujących agentów kolidują ze sobą.

Przykład implementacji

Autonomiczne światła drogowe: Sterowniki sąsiednich skrzyżowań na bieżąco negocjują długość cykli świetlnych. Dążą do zminimalizowania łącznych przestojów aut (kary f_{ij}) w całej dzielnicy.

Rywalizacja: Równowaga Nasha

Cel: Osiągnięcie matematycznej stabilności w środowiskach wieloagentowych charakteryzujących się konfliktem interesów.

$$V_i(\pi_i^*, \pi_{-i}^*) \geq V_i(\pi_i, \pi_{-i}^*) \quad \forall i, \forall \pi_i$$

Analiza zmiennych równania:

- V_i : Funkcja użyteczności agenta i .
- π_i^* : Wypracowana, zoptymalizowana strategia agenta i .
- π_{-i}^* : Wektor strategii wszystkich pozostałych agentów w systemie.
- π_i : Jednostronna dewiacja (dowolna, samowolnie zmieniona strategia agenta i).

Interpretacja: System jest stabilny, gdy żadnemu agentowi nie opłaca się jednostronnie zmienić swojego planu działania, o ile pozostali agenci nie zmieniają swoich.

Przykład implementacji

Zautomatyzowane rynki (np. Smart Grids): Autonomiczne algorytmy negocjują stawki dystrybucji zasobów. Układ zastyga w równowadze, gdy jakakolwiek samodzielna zmiana algorytmu licytującego przyniesie agentowi wyłącznie stratę.

Rywalizacja: reguła minimax w grach o sumie zerowej

Cel: Maksymalizacja minimalnej oczekiwanej wypłaty w środowiskach stricte antagonistycznych.

$$V_1^* = \max_{\pi_1} \min_{\pi_2} R_1(\pi_1, \pi_2)$$

Analiza zmiennych równania:

- V_1^* : Gwarantowane dolne ograniczenie wypłaty dla wariantu pesymistycznego.
- $\min_{\pi_2}$: Operator modelujący zachowanie racjonalnego oponenta, którego jedynym celem jest minimalizacja zysku agenta 1.
- $\max_{\pi_1}$: Optymalna odpowiedź agenta 1, mająca na celu wybór strategii chroniącej przed najgorszym rozkładem użyteczności zdefiniowanym przez operator minimujący.
- R_1 : Funkcja zysku agenta 1.

Przykład implementacji

Planowanie trajektorii obronnej: Nawigacja autonomicznych jednostek przy założeniu perfekcyjnej strategii jednostki przechwytyjącej.

Wieloagentowe algorytmy ewolucyjne i genetyczne (MAGA)

Cel: Rozwiązywanie problemów optymalizacyjnych o ogromnej złożoności poprzez symulowanie procesu doboru naturalnego.

Mechanika algorytmów MAGA (Multi-Agent Genetic Algorithms):

- **Populacja strategii:** Zamiast wyliczać jedną strategię, system generuje całą populację losowych planów działania dla agentów.
- **Krzyżowanie (Crossover) i Mutacja:** Najlepiej punktujące strategie (te o najwyższej użyteczności V) są krzyżowane ze sobą, a część ich parametrów ulega losowej mutacji, tworząc nową generację agentów.
- **Selekcja naturalna:** Przetrywają wyłącznie te strategie, które najlepiej rozwiązują globalny problem wieloagentowy.

Zastosowanie

Optymalizacja harmonogramów zautomatyzowanych linii produkcyjnych, gdzie liczba kombinacji maszyn przekracza możliwości obliczeniowe klasycznej matematyki.

Metryki oceny systemów (MAS)

Ewaluacja układów rozproszonych wymaga zastosowania rygorystycznych metryk uwzględniających uwarunkowania topologiczne:

- **Narzut komunikacyjny (Communication Overhead):** Całkowity wolumen pakietów danych wymienianych w systemie niezbędny do osiągnięcia konwergencji.
- **Szybkość zbieżności (Convergence Rate):** Czas (lub liczba iteracji decyzyjnych) wymagany do asymilacji lokalnych stanów węzłów do postaci globalnego ekstremum optymalizacyjnego.
- **Odporność topologiczna (Robustness):** Tolerancja układu na awarię węzłów oraz na ograniczenia wynikające z częściowej obserwowalności środowiska (POMDP – *Partially Observable Markov Decision Process*).

Najważniejsza idea

Systemy wieloagentowe składają się z autonomicznych agentów, którzy podejmują decyzje lokalnie, a zachowanie całego systemu wynika z ich współpracy, rywalizacji lub adaptacyjnego uczenia się.

Współpraca

- konsensus i synchronizacja stanów,
- rozproszona optymalizacja DCOP,
- algorytmy roju: separacja, wyrównanie, spójność,
- zastosowania: robotyka rojowa, routing, drony, systemy magazynowe.

Rywalizacja i adaptacja

- równowaga Nasha jako punkt stabilności strategii,
- minimax w grach o sumie zerowej,
- MARL: uczenie polityk przez interakcję ze środowiskiem,
- MAGA: ewolucyjna selekcja najlepszych strategii.

Wniosek końcowy

Klasyczne metody analityczne dobrze opisują małe i znane środowiska, natomiast w złożonych systemach praktycznych większe znaczenie mają podejścia heurystyczne i uczące się, zwłaszcza MARL oraz algorytmy ewolucyjne.

Dziękujemy za uwagę.

- Y. Shoham, K. Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.
- https://medium.com/@diwakarkumar_18755/building-multi-agent-systems-with-langgraph-and-ollama-architectures-concepts-and-code-383d4c01e00c
- R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.
- <https://www.ampcome.com/post/what-is-a-multi-agent-ai-system>
- <https://www.nvidia.com/en-us/glossary/multi-agent-systems/>
- <https://milvus.io/ai-quick-reference/what-algorithms-are-used-in-multiagent-systems>
- https://www.researchgate.net/figure/Three-rules-of-Boids-algorithm-a-collision-avoidance-b-alignment-and-c_fig1_369711253

Algorytmy współpracy i rywalizacji w systemach wieloagentowych

Jakub Jaśków, Justyna Ziemichód

1 sierpnia 2026